

Instituto Politécnico Nacional

Escuela Superior de Cómputo

Web Client and Backend Development Frameworks.

Investigación: el patrón de diseño DAO (Data Access Object)

Profesor: Dr. José Asunción Enríquez Zárate

Alumno: Angel Frausto Robles

Boleta: 2019630177

id634296@gmail.com

7CM2

19 de septiembre de 2026

Índice

1. Introducción	2
2. Definición formal	2
3. Estructura y componentes	2
4. Ejemplo en Java	3
5. Ventajas y desventajas	4
6. Conclusión	4
7. Referencias Bibliográficas	5

Índice de figuras

1. Estructura del patrón DAO aplicada al ejemplo Carrera 2

Índice de tablas

1. Ventajas y desventajas del patrón DAO 4

1. Introducción

El patrón DAO (Data Access Object) aparece en el catálogo Core J2EE Patterns, publicado por Sun Microsystems a partir del trabajo de Alur, Crupi y Malks [1]. Ataca un problema concreto de las aplicaciones empresariales en Java: la lógica de negocio termina mezclada con el código que habla directamente con la base de datos, y cualquier cambio de motor o de esquema obliga a tocar clases que no debieran saber nada de SQL.

El patrón separa esas dos responsabilidades. La lógica de negocio pide datos a través de una interfaz fija; quien implementa esa interfaz decide cómo conseguirlos, ya sea con JDBC contra MySQL, con un archivo o con una llamada a otro servicio. Ese acomodo explica por qué el patrón sigue vigente mucho después de J2EE: aparece bajo el mismo nombre en Spring, en Hibernate y en cualquier proyecto que necesite aislar la persistencia del resto de la aplicación [2].

El ejemplo que recorre este documento es un recurso llamado **Carrera**: primero se define la clase que representa el dato y después la clase que lo guarda y lo recupera de MySQL.

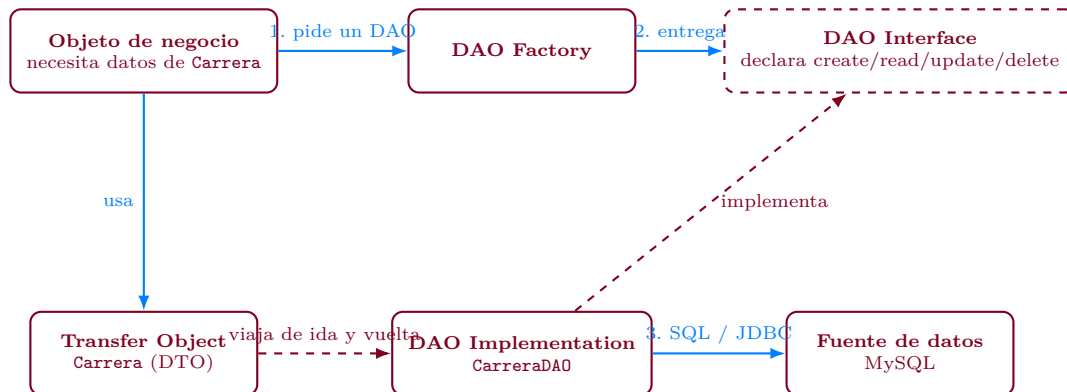
2. Definición formal

Core J2EE Patterns define al Data Access Object como el objeto responsable de abstraer y encapsular todo el acceso a una fuente de datos [1]. El objeto de negocio que necesita esos datos nunca llama directamente al manejador de base de datos: llama al DAO, y el DAO es el único que conoce el detalle de cómo se guarda y se recupera la información.

Esa frontera es la misma que separa una interfaz de su implementación. Mientras el contrato del DAO no cambie, la aplicación puede migrar de MySQL a otro motor, o incluso a una fuente de datos que no sea una base de datos relacional, sin que el resto del código note la diferencia [2].

3. Estructura y componentes

La figura 1 ubica las cuatro piezas del patrón sobre ese mismo ejemplo.



El cliente solo conoce la interfaz y el DTO; nunca ve el SQL ni sabe qué motor hay detrás

Figura 1: Estructura del patrón DAO aplicada al ejemplo Carrera

DAO Interface. Declara las operaciones disponibles (crear, leer, actualizar, eliminar) sin decir cómo se ejecutan. Es el contrato que ve el resto de la aplicación.

DAO Implementation. Clase concreta que cumple ese contrato contra un motor de base de datos particular. **CarreraDAO** es esta pieza: abre la conexión JDBC, arma las consultas preparadas y las ejecuta.

Transfer Object (DTO). Objeto simple que carga los datos de ida y vuelta entre el DAO y quien lo usa, sin ninguna lógica propia. **Carrera** cumple este papel: solo tiene atributos, constructores y accessors/mutators.

DAO Factory. Clase que decide, en tiempo de ejecución, qué implementación concreta entregar, para que quien la pide nunca escriba el nombre de la clase concreta en su propio código. Es la pieza que este ejemplo no necesita, porque con un solo motor de base de datos no hace falta elegir entre implementaciones.

4. Ejemplo en Java

El DTO y la implementación concreta retoman el ejemplo de **Carrera** presentado antes.

Código 1: Transfer Object: Carrera.java (fragmento)

```
1 public class Carrera {
2     private int idCarrera;
3     private String nombreCarrera;
4     private String descripcionCarrera;
5
6     public Carrera() { }
7
8     public void setIdCarrera(int idCarrera) { this.idCarrera = idCarrera; }
9     public int getIdCarrera() { return this.idCarrera; }
10    public void setNombreCarrera(String n) { this.nombreCarrera = n; }
11    public String getNombreCarrera() { return this.nombreCarrera; }
12    public void setDescripcionCarrera(String d) { this.descripcionCarrera = d; }
13    public String getDescripcionCarrera() { return this.descripcionCarrera; }
14 }
```

Código 2: DAO Implementation: CarreraDAO.java (fragmento con create y read)

```
1 public class CarreraDAO {
2     private static final String SQL_INSERT =
3         "INSERT INTO Carrera (nombreCarrera, descripcionCarrera) VALUES (?, ?)";
4     private static final String SQL_SELECT =
5         "SELECT * FROM Carrera WHERE idCarrera = ?";
6
7     private Connection conexion;
8
9     private void obtenerConexion() throws SQLException {
10        Class.forName("com.mysql.cj.jdbc.Driver");
11        conexion = DriverManager.getConnection(
12            "jdbc:mysql://localhost:3306/MisCursos", "root", "tu_password");
13    }
14
15    public void create(Carrera c) throws SQLException {
16        obtenerConexion();
17        PreparedStatement ps = conexion.prepareStatement(SQL_INSERT);
18        ps.setString(1, c.getNombreCarrera());
19        ps.setString(2, c.getDescripcionCarrera());
20        ps.executeUpdate();
21        ps.close();
22        conexion.close();
23    }
24
25    public Carrera read(Carrera c) throws SQLException {
26        obtenerConexion();
27        PreparedStatement ps = conexion.prepareStatement(SQL_SELECT);
28        ps.setInt(1, c.getIdCarrera());
29        ResultSet rs = ps.executeQuery();
30        Carrera resultado = null;
31        if (rs.next()) {
32            resultado = new Carrera();
33            resultado.setIdCarrera(rs.getInt("idCarrera"));
34            resultado.setNombreCarrera(rs.getString("nombreCarrera"));
35            resultado.setDescripcionCarrera(rs.getString("descripcionCarrera"));
36        }
37        rs.close();
38        ps.close();
39        conexion.close();
40        return resultado;
41    }
42 }
```

La versión completa agrega los métodos restantes del CRUD (**update**, **delete**, **readAll**) y el manejo de excepciones, siguiendo el mismo patrón que **create** y **read**.

Los dos códigos siguientes son ilustrativos: agregan la interfaz y la fábrica que el ejemplo anterior no necesitaba, porque con un solo motor de base de datos no hace falta elegir entre implementaciones. Se agregan aquí para completar los cuatro componentes del patrón.

Código 3: DAO Interface ilustrativa

```

1 public interface DAO<T> {
2     void create(T objeto) throws SQLException;
3     T read(T objeto) throws SQLException;
4     List<T> readAll() throws SQLException;
5     void update(T objeto) throws SQLException;
6     void delete(T objeto) throws SQLException;
7 }

```

Código 4: DAO Factory ilustrativa

```

1 public class DAOFactory {
2     public static DAO<Carrera> obtenerCarreraDAO() {
3         return new CarreraDAO();
4     }
5 }

```

Con esas dos piezas agregadas, **CarreraDAO** pasaría a implementar **DAO<Carrera>** en vez de no declarar ninguna interfaz. Quien lo usa, en vez de escribir **new CarreraDAO()**, se lo pediría a la fábrica.

5. Ventajas y desventajas

Ventajas	Desventajas
Separa la lógica de negocio del código de acceso a datos.	Agrega clases e interfaces adicionales; en un proyecto muy pequeño puede ser más estructura de la que hace falta.
Facilita cambiar de motor de base de datos sin tocar el resto de la aplicación, porque solo cambia la implementación concreta.	Si el DTO y el DAO no se diseñan con cuidado, es fácil filtrar detalles de la base de datos (nombres de columnas, tipos SQL) hacia capas superiores.
Concentra el SQL en un solo lugar, más fácil de revisar y de probar.	Cada operación nueva (una consulta distinta, un filtro) obliga a agregar un método a la interfaz y a todas sus implementaciones.
Permite sustituir la implementación real por una de prueba sin usar una base de datos real.	

Tabla 1: Ventajas y desventajas del patrón DAO

6. Conclusión

El **CarreraDAO** de este ejemplo ya tiene la pieza central del patrón: quien lo usa nunca escribe SQL, solo llama a **create**, **read** o **delete** y deja que el DAO se encargue de la conexión y de las consultas preparadas. Lo que le falta, la interfaz y la fábrica, no se nota mientras el proyecto tenga un solo motor de base de datos y una sola implementación. Se volvería necesario en cuanto apareciera una segunda implementación real, por ejemplo una versión de prueba sin base de datos para los exámenes unitarios, o un cambio de motor de base de datos ya avanzado el proyecto.

La utilidad del patrón está en dejar preparado el punto donde la aplicación va a necesitar cambiar más adelante, aunque ese cambio no exista todavía el primer día. Separar responsabilidades cuesta algunas clases de más al principio; a cambio, ese cambio futuro se resuelve escribiendo una clase nueva en vez de reescribiendo la aplicación entera.

Angel Frausto Robles

7. Referencias Bibliográficas

- [1] Deepak Alur, John Crupi y Dan Malks. *Core J2EE Patterns: Best Practices and Design Strategies*. 2.^a ed. Prentice Hall, 2003.
- [2] Oracle Corporation. *Design Patterns: Data Access Object*. URL: <https://www.oracle.com/java/technologies/dataaccessobject.html> (visitado 19-09-2026).